C# 程序设计 · 大作业研究报告

俄罗斯方块的设计与实现

——基于 .NET 10 + WinForms 的分离式架构、AI 自动玩家与健壮性设计

项目名称：俄罗斯方块（Tetris）

运行环境：Windows 10 / 11 • .NET SDK 10.0 • WinForms

小组成员与分工

学号	姓名	分工	贡献比
【学号 1】	【姓名 1】	组长。核心规则与引擎（Tetris.Core）、发布与集成、报告数据实测	40%
【学号 2】	【姓名 2】	WinForms 界面、输入（DAS/ARR）、XML 存档与异常处理	32%
【学号 3】	【姓名 3】	启发式 AI（评估 / 落点搜索 / 指令状态机）与对应单元测试	28%

（学号、姓名请按实际情况填写；贡献比三人不同，合计 100%）

2026 年 10 月 30 日

摘　要

本文报告了以 C# 与 .NET 10 实现的俄罗斯方块大作业。程序采用 WinForms 界面，严格实现 12 列 × 16 行棋盘、七种方块的移动与双向旋转、等级加速、暂停与结束判定，分数按“一次消 k 行得 100 × k”计算并以十六进制显示，历史最高分以 XML 保存在用户漫游目录。工程按“逻辑与界面分离”拆成五个项目：Core 不引用任何界面技术，状态可脱离窗口独立创建并深拷贝，界面只通过五个接口读状态、下发命令。扩展功能选择 AI 自动玩家：四项棋盘特征的线性评估配合 1-ply / 2-ply 落点搜索，在真引擎上 10 个种子的平均成绩为 68730 分、687 行，10 局均未被顶死。88 个 xUnit 用例覆盖碰撞、旋转踢墙、消行计分、引擎时序、存档异常与 AI 不变量，全部通过。报告给出逻辑帧 0.44 μs、深拷贝 0.17 μs、单次落点搜索 0.21 ms 等数据，并复盘了落点盲区、2-ply 加权失效等问题。

关键词：俄罗斯方块；C#；WinForms；关注点分离；启发式搜索；异常处理

第一章　引言

1.1　项目背景

俄罗斯方块由阿列克谢·帕基特诺夫于 1984 年设计，规则一句话就能说完：让下落的四格方块填满整行并消除。但它内部包含了一个完整软件系统的要素——二维数组上的碰撞检测、带边界条件的几何变换、定时驱动的状态推进、可复现的随机序列，以及“输入 → 状态 → 渲染”的主循环。正因为它足够小，每条设计决策的后果都看得见，我们把它当成一次工程练习，而不只是算法作业。

课程指定 .NET 10（LTS）。C# 在这里既有成熟的 WinForms 桌面栈，又有可空引用类型、模式匹配等现代特性，很适合把“界面”和“逻辑”切成两块互不依赖的代码：逻辑层能被单元测试直接驱动，界面层只负责画状态、翻译按键。

1.2　游戏类型与设计目标

本作品是单机、单人的落块消除类游戏：七种方块自顶部落下，可左右移动与双向旋转，触底或被阻挡后固定；整行填满即消除，消行累积后升级、重力加快，新方块在出生位置即碰撞则结束。三个目标：

（1）玩法正确、手感可接受：旋转、踢墙、连消、加速不能“错一拍”，长按左右要有稳定的连发节奏。

（2）架构可讲清、可测试：规则与渲染彻底分离，状态能独立创建与复制，核心逻辑有有效的单元测试。

（3）异常不崩溃：存档缺失、损坏、被占用、只读，窗口缩放、失焦、快速连按，都不能假死或闪退。

1.3　报告结构

第二章说明需求理解与歧义处理；第三章讲模块划分、核心算法、界面与输入设计及扩展功能 AI；第四章给出效果、排查过程与量化数据；第五章复盘协作与 AI 使用；第六章总结。

第二章　需求理解与假设

2.1　需求的落地口径

任务书写清了“做什么”，没写清“做到什么程度”。我们先把需求翻译成可验证的口径（表 2-1），再解释六处歧义。

表 2-1　需求条目与实现口径对照

需求条目	我们的实现口径
棋盘 12 列 × 16 行	GameConfig.BoardWidth=12、BoardHeight=16；坐标 [x, y]，x 为列、y 为行、y=0 在顶部
七种方块与旋转	手写 4 个旋转态数据，双向旋转，简化 SRS 踢墙（见 2.2 第 1 条）
计分 100 × k	ScoreFor(k) 返回 100×k，无等级倍率、无硬降分、无 T-spin 奖励
等级加速	每消 10 行升 1 级；13 档重力 800→50 ms（见 2.2 第 2 条）
分数十六进制	界面统一格式化为 0x 前缀大写十六进制；内部与存档一律十进制
最高分 XML	单条 HighScore 记录，%AppData%\TetrisCourse\highscore.xml，启动即读、破纪录即写
逻辑与界面分离	Tetris.Core 不引用任何界面技术；GameState 可独立 new、可 Clone 深拷贝
单元测试不少于 5 个	实际交付 88 个 xUnit 用例，覆盖碰撞、旋转、消行、计分、引擎时序、存档与 AI

2.2　歧义的处理方式

（1）旋转是否必须完整 SRS。我们实现简化 SRS：只保留一档踢墙，JLSTZ 依次尝试 (0,0)、(-1,0)、(1,0)、(-2,0)、(2,0)、(0,-1)，I 用另一套顺序。理由：12 列比标准 10 列宽，贴墙旋转机会更少；完整表要 20 条人工核对的数据，验证成本高于收益。

（2）升级曲线未给定。取 13 档：800、720、630、550、470、380、300、220、160、120、90、70、50 ms，之后维持 50 ms。

（3）结束判据未定义。取“新方块在出生位置即碰撞”：纯逻辑判据，可在测试里精确构造，也不随窗口尺寸变化。

（4）暂停边界未定义。暂停期间只接受“继续”，逻辑帧不增长；失焦或最小化自动暂停并清空按键，避免补发积压的移动。

（5）“下一个方块”数量未规定。实现为 3 个预览并排显示，队列不足时留空。

（6）“人机对战或观战演示”解释为：同一棋盘、同一引擎实例，F3 在玩家与 AI 间交接控制权；AI 期间玩家移动被忽略，但暂停有效，切回时方块不跳变。

2.3 明确的非目标

不实现双人、回放与网络，仅支持 Windows。三门扩展里 AI 与“架构分离、状态可复制、测试有效”结合最紧：它要求规则完全可推演、状态可克隆、界面只读状态，任何耦合都会被放大成 bug；双人与网络会把工期拉长到无法保证质量。

第三章　设计与实现

3.1 模块划分与组装点

解决方案拆成五个项目与一个测试工程，依赖严格单向：App → Persistence → Core，App → Core，AI → Core，tests → Core / AI；反向引用视为缺陷。

表 3-1　项目划分与代码规模（不含生成代码）

项目	行数	职责
Tetris.Core	1335	状态、规则、引擎、全部接口，无任何界面依赖
Tetris.App	1297	WinForms 界面、自绘渲染、输入路由、存档接线
Tetris.AI	585	评估函数、落点搜索、指令状态机
Tetris.Persistence	192	最高分 XML 读写
tests/Tetris.Core.Tests	1637	88 个用例与两套测试替身

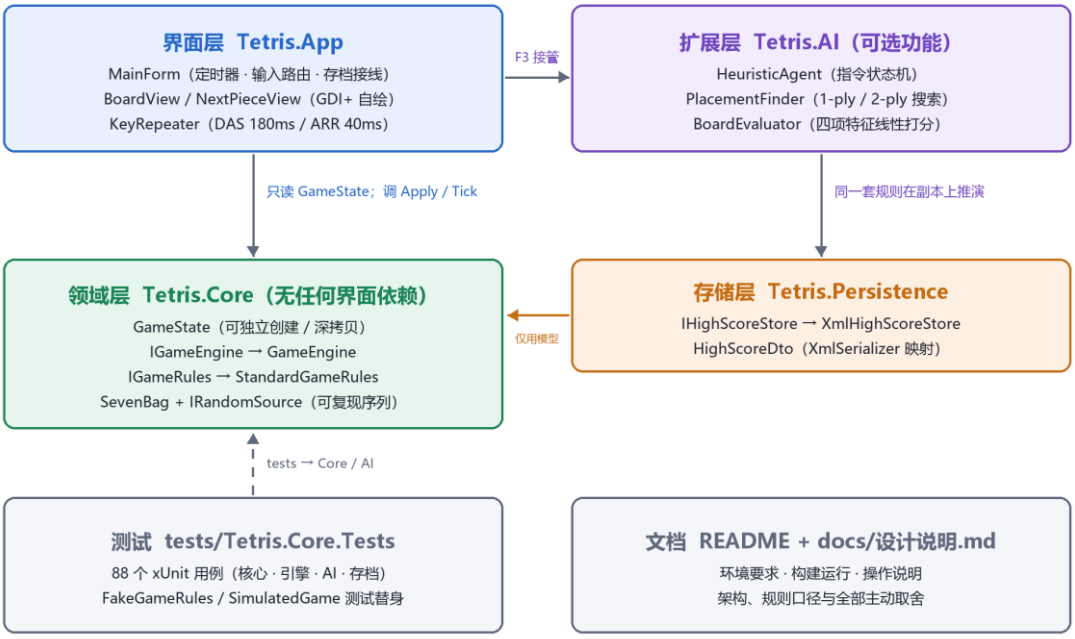


图 3-1　分层架构、依赖方向与唯一组装点

值得注意的是“唯一组装点”：全工程只有 AppBootstrap 一处 new 具体类型，由它创建引擎、规则、随机源与存档，再以接口交给界面。MainForm 只认识 IGameEngine，AI 只接收 IGameRules。代价是前期多写五个接口，收益是三人可并行、测试可替换任意一层。

3.2　五个契约与共享数据

表 3-2　五个接口契约

接口	实现	调用方	职责
IGameRules	StandardGameRules	引擎、AI、测试	碰撞、下落距离、移动、旋转踢墙、锁定消行
IGameEngine	GameEngine	界面	开局、接受命令、推进一帧、读状态
IHighScoreStore	XmlHighScoreStore	界面、测试	读写最高分，失败不抛异常
IPlayerAgent	HeuristicAgent	界面	每帧产出一条 GameCommand
IPlayerAgentDebug	HeuristicAgent	界面（可选）	暴露候选落点与选中项，用于画热力图

共享数据只有五个：GameState、Tetromino、GameCommand、GameConfig、PieceShape。GameState 是唯一被三方触碰的对象：引擎读写、界面只读、AI 必须先 Clone。这条规则写进注释，也写进测试。

3.3 核心算法

3.3.1 碰撞与下落距离

碰撞按四个实心格逐一判定：横向越界算撞墙，向下越界算撞地板，棋盘内已占据算叠住，y < 0 的格子允许存在。最后一条决定锁定时的丢弃规则。

```
public bool Collides(GameState state, Tetromino piece)
{
    if (state is null || piece is null) return true;   // 永不抛异常
    foreach (var cell in piece.Cells)
    {
        if (cell.X < 0 || cell.X >= state.Width) return true;   // 左右墙
        if (cell.Y >= state.Height) return true;                // 地板
        if (cell.Y >= 0 && state.Board[cell.X, cell.Y].HasValue) return true;
    }
    return false;   // y < 0 的格子在棋盘外，允许存在
}
```

下落距离复用同一套判断逐格试探，上限 Height + BoxSize 只为防死循环。碰撞是纯查询，因此“能否移动、旋转、落几格”共用一份实现。

3.3.2 旋转与踢墙

旋转只改朝向编号，再由形状数据给出新坐标，随后按踢墙表依次尝试偏移，第一个不碰撞的位置生效；全失败则返回 false 且状态不变。

```
private static readonly (int Dx, int Dy)[] JlstzKicks =
    { (0, 0), (-1, 0), (1, 0), (-2, 0), (2, 0), (0, -1) };
private static readonly (int Dx, int Dy)[] IKicks =
    { (0, 0), (-2, 0), (2, 0), (-1, 0), (1, 0), (0, -1) };

public bool TryRotate(GameState state, int direction)
{
    var rotated = state.Active.With(
        rotation: state.Active.Rotation + direction);
    var kicks = state.Active.Kind == PieceKind.I ? IKicks : JlstzKicks;
    foreach (var kick in kicks)
    {
        var candidate = rotated.With(
            x: rotated.X + kick.Dx,
            y: rotated.Y + kick.Dy);
        if (Collides(state, candidate)) continue;
        state.Active = candidate;   // 只在这里写状态
        return true;
    }
    return false;                   // 全部失败：状态不变
}
```

3.3.3 锁定、消行与计分

锁定三步：写盘（探出顶部的丢弃）、从下往上压实、清空顶部空行，最后计分并重算等级。压实一趟扫完，O(H × W)。

```
var cleared = 0;
var write = state.Height - 1;
for (var read = state.Height - 1; read >= 0; read--)
{
    if (state.IsRowFull(read)) { cleared++; continue; }
    if (write != read)
        for (var x = 0; x < state.Width; x++)
            state.Board[x, write] = state.Board[x, read];
    write--;
}
for (; write >= 0; write--)          // 顶部补空行
    for (var x = 0; x < state.Width; x++)
        state.Board[x, write] = null;

state.Score += state.Config.ScoreRule.ScoreFor(cleared, state.Level);
state.LinesCleared += cleared;
state.Level = linesPerLevel > 0 ? state.LinesCleared / linesPerLevel + 1 : 1;
```

刻意的职责划分：LockActive 只管锁定、消行、计分，不生成下一个方块——否则 AI 在副本上推演就多一次出块，结果会被污染。

3.3.4 重力与逻辑帧

逻辑帧固定 1/60 秒，重力用毫秒累加器：每帧加 16.67 ms，攒够当前等级的间隔就下落一格，贴底则锁定出新块。用时间累加而非帧计数，是为了让 0.5× / 1× / 2× 只缩放逻辑时间，不改变按键响应。

```
public void Tick()
{
    if (_state.Status != GameStatus.Running) return;   // 暂停时 Frame 不增长
    _state.Frame++;
    _accumulatorMs += FrameMs;                          // 1000.0 / 60
    if (_accumulatorMs < _config.GravityMsFor(_state.Level)) return;
    _accumulatorMs = 0;                                 // 直接清零，不减去间隔
    var fallen = _state.Active.With(y: _state.Active.Y + 1);
    if (!_rules.Collides(_state, fallen)) { _state.Active = fallen; return; }
    LockAndSpawn();                                     // 贴底：锁定并出新块
}
```

累加器“直接清零”而不减去间隔，是有意取舍：减法长期精度更好，但会让高等级节奏轻微抖动。原因写在注释里，避免后来者当成 bug 改回去。

3.3.5 随机序列

出块用 7-bag：七种各一个，Fisher-Yates 洗牌后逐个取出，空袋再装。随机源可注入，默认 xorshift32；不用 System.Random，因为它不保证跨版本序列一致。

```
_state ^= _state << 13;
_state ^= _state >> 17;
_state ^= _state << 5;
return (int)(_state % (uint)maxExclusive);
```

3.4 状态管理

GameState 是“可独立创建、可复制”的落点：公开构造只依赖 GameConfig，私有拷贝构造深拷贝棋盘、当前方块、预览队列、分数、等级、消行、状态与帧号，因此 AI 每块克隆几十次也不影响棋局。界面侧约定只读：每帧把状态引用交给 BoardView，按指纹决定是否重绘（指纹不含帧号，否则每帧都变），实测暂停时重绘从每秒 66 次降到 0。

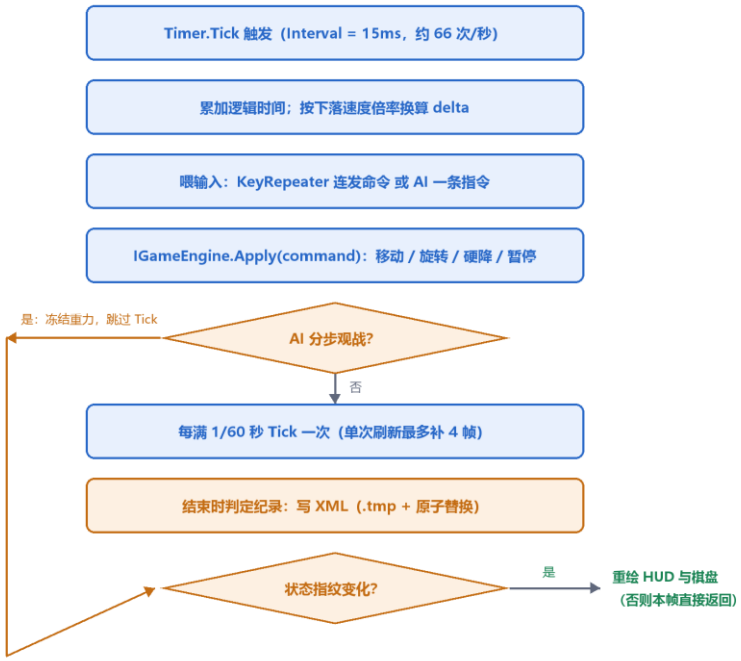


图 3-2 一帧之内的处理流程

3.5 界面设计

界面是两栏：左侧棋盘随窗口自适应，右侧固定 236 像素。窗口用 TableLayoutPanel 组织，棋盘按可用区域重算格子大小并居中；最小尺寸限 720 × 560，避免格子过小与布局错乱。



图 3-3 界面布局示意（按 860 × 660 的设计尺寸绘制）

配色为深色：背景 #101016、棋盘 #1A1A22、网格 #2C2C3A，分数用青色 #78D2FF。方块颜色从 PieceShape.ColorHex 读取，换肤只需改形状数据。棋盘控件还画暂停/结束遮罩与 AI 候选热力图，候选为空时不留幽灵方块。

3.6 输入处理

输入分两路：一次性命令（旋转、硬降、暂停）在 ProcessCmdKey 按下当帧下发，按住不连发；可重复命令（左右、软降）登记到 KeyRepeater，由它按 DAS 180 ms / ARR 40 ms 每帧前决定发几条。这样“按一下走一格”与“按住连走”共用一套时钟，不依赖 Windows 自动重复。

细节：先裁掉 keyData 的修饰位，保证按下与松开是同一个键；第二次 KeyDown 视为系统重复并吞掉；左右同按时最后按下的方向优先，被顶掉的方向退到排队，松手后 40 ms 内接管；暂停、失焦、最小化统一清空按键表。

3.7 高分存档

存档契约为 Load / Save 永不抛异常，失败翻译成四种状态：不存在 → Missing；乱码、根节点不对、负数、超过 64 KB → Corrupted；路径是目录、被独

占、无权限 → Unreadable；其余 Ok，界面因此不需要 try/catch。写入走“同目录写 .tmp → Flush → File.Move 覆盖”的原子替换，失败时返回 false 并在侧栏提示未保存。

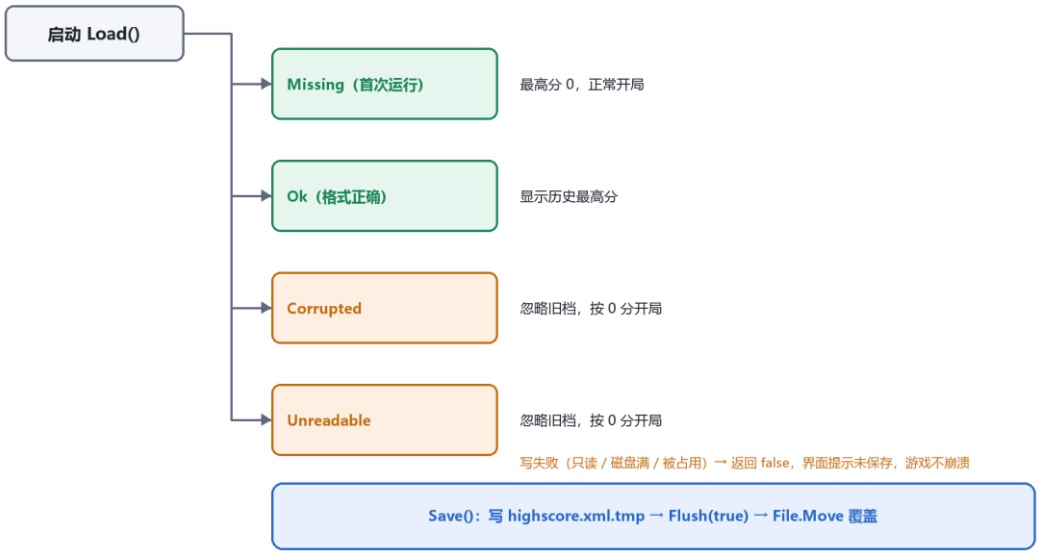


图 3-4　存档读写状态机

```
using (var stream = new FileStream(
    tempPath, FileMode.Create, FileAccess.Write, FileShare.None))
{
    Serializer.Serialize(stream, dto);
    stream.Flush(flushToDisk: true);      // 先落盘，避免“改名成功但内容是空的”
}
File.Move(tempPath, _path, overwrite: true);   // 同目录覆盖 = 原子替换
return true;
```

3.8　扩展功能：启发式 AI 自动玩家

AI 分三层，每层只依赖下一层：BoardEvaluator 是纯函数，只算四项指标；PlacementFinder 只依赖 IGameRules，用 Clone 推演落点并打分；HeuristicAgent 是状态机，每帧只产出一条 GameCommand。三层可独立测试。

评估函数为 0.76 × 消行 − 0.51 × 总高度 − 0.36 × 空洞 − 0.18 × 凹凸度：消行是唯一正收益，空洞惩罚仅次于它，凹凸度抑制搭塔。搜索对四个旋转态 × 包围盒左列的每个合法位置试放：Clone、落底、锁定消行、打分，单块候选平均约 42 个。

2-ply 的用法比较特殊：不给所有候选加权，而是先用 1-ply 定出最优分，只在“与最优分相差不超过 0.15”的候选里用“当前分 + 0.4 × 下一块最优分”裁

决。候选的 Score 始终是纯 1-ply 分，热力图尺度不失真。为什么不做全量加权见 4.2.2。

```
public GameCommand NextCommand(GameState state)
{
    if (_plan is null || _plan.Kind != state.Active.Kind) Replan(state);
    if (_plan is null) return HardDropAndForget();       // 无合法落点：砸下去，让引擎判负
    if (++_framesOnPiece > MaxFramesPerPiece) return HardDropAndForget();

    var plan = _plan!;
    if (!_rotationAbandoned && state.Active.Rotation != plan.Rotation)
        return GameCommand.RotateCw;
    if (state.Active.X < plan.X) return GameCommand.MoveRight;
    if (state.Active.X > plan.X) return GameCommand.MoveLeft;
    return HardDropAndForget();                          // 位置与朝向都对：硬降
}
```

三条兜底保证不卡死：旋转连续 4 帧没到位就放弃；同一块超过 120 帧强制硬降；连续 3 次重规划无进展也强制硬降。硬降后计划清空，下一块重新搜索。

3.9 第三方库

运行时零第三方依赖，只用基类库：WinForms、XmlSerializer、Stopwatch。测试用 xUnit 2.9.3、Microsoft.NET.Test.Sdk 17.14.1 与 coverlet.collector 6.0.4。五个对象的手工组装写在 37 行 AppBootstrap 里，比依赖注入容器更容易讲清。

第四章　分析

4.1 实现效果

核心玩法完整可玩：移动、双向旋转、软降、硬降、暂停、重开、消行、升级、加速、结束判定全部实现。AI 可用 F3 在任意时刻接管或交还，用 F5 在“分步”（110 ms 一条指令、落块后停 140 ms）与“极速”两种节奏间切换。

表 4-1　实现效果与验证结果

验证项	结果
Release 编译	5 个项目 + 测试工程，0 警告 0 错误
单元 / 集成测试	88 个用例全部通过，整轮约 1.25 秒
AI 长跑（10 个种子，每局上限 10000 帧）	平均 68730 分 / 687 行，10 局全部跑满上限仍未顶死
存档异常	缺失 / 损坏 / 超大 / 只读 / 被占用 / 路径是目录，六种场景均不崩溃

窗口缩放与失焦	最小 720 × 560，缩放不闪；失焦自动暂停
发布产物	dotnet publish（win-x64 框架依赖）11 个文件、约 0.3 MB

4.2 遇到的问题与解决过程

下面四个问题按发现顺序写，每个都附定位方式。这一节最想说明的是：没有一个 bug 是被“读代码”发现的。

4.2.1 落点枚举的左墙盲区

AI 接上真引擎后，长跑的平均消行明显低于预期，观战时能看到竖放的 I 从不进最左两列。定位过程：先写“所有候选都在棋盘内且已落底”的不变量测试——通过；再写“竖 I 应覆盖每一列”——失败。原因是枚举从 x = 0 起，而包围盒宽 4 格、竖 I 的实心列在盒内第 2 列，最左可落位置其实是 x = −2。改为从 −(BoxSize − 1) 起后盲区消失，并补了两条回归用例。教训是：不变量只能证明“没越界”，证明不了“没漏掉”。

4.2.2 2-ply 全量加权反而更快顶死

第一版 2-ply 给每个候选算“当前分 + 权重 × 下一块最优分”再取最大，结果第 380 帧左右就顶死。回放显示加权后的分数普遍低于未加权，AI 会为了“下一块能消一行”先把棋盘堆成塔。改成只在并列最优（差距不超过 0.15）的候选里裁决后，平均分从 59330 提升到 68730，10 局全部跑满上限。

4.2.3 分步观战被重力抢先锁定

分步档把指令间隔放宽到 110 ms，本意是让人看清“旋转 → 平移 → 落点”。但高等级重力最快 50 ms 一格，方块会在指令发完前被锁定在错误列，观战像 AI 在乱放。修复是在分步档冻结重力，方块只随指令移动；再补 2.5 秒停滞兜底，同一块长时间没有成功指令就替它硬降。

4.2.4 手工旋转态比严格旋转左移一列

核对旋转态时发现，T/S/Z/J/L 的四个旋转态相对严格旋转整体左移了一列（I 的偏移为 0，O 四态相同），玩家顺时针旋转时会感觉方块“往左跳一格”。我们没有改：碰撞、踢墙、消行都不依赖这个偏移，AI 按列计算也不受影响，改数据会让已录的实测与基准失效。这一条写进了设计说明的“主动取舍”。

4.2.5 存档异常的六个场景

手工构造六种异常：删除文件、写入乱码、改根节点、塞一个 70 MB 的文件、把路径换成同名目录、用另一进程独占打开。前四种被翻译成 Missing 或 Corrupted，后两种 Unreadable，界面各显示不超过 16 字的提示，游戏照常开局。做法不巧妙，只是先把“失败清单”列全，再为每条写一个用例。

4.3 性能优化

先量再改，一共四处。重绘节流：按状态指纹判断后，稳定局面的重绘降为 0。补帧上限：Timer 间隔 15 ms、逻辑帧 16.67 ms，2× 速度下一次回调可能推进两帧；若按实际时间差补齐，拖动窗口后会一次推进几十帧，因此把单次刷新最多补 4 帧写死。AI 每块只搜索一次，其余帧只读计划；2-ply 只搜并列最优候选。

4.4 量化数据

数据在同一台机器（Windows 11、.NET 10.0.13、Release）测得，测试代码与正式代码共用同一套规则与引擎，种子固定为 1 到 10。

表 4-2 核心性能数据

指标	数值	测量方式
逻辑帧耗时（纯重力）	0.44 μs / 帧	20 局共 69041 帧，Stopwatch 计时
逻辑帧耗时（含命令）	2.00 μs / 帧	20 局，每帧一条命令 + 一次 Tick
GameState.Clone	0.17 μs / 次	20 万次深拷贝
AI 落点搜索（1-ply）	0.069 ms 均值 / 0.49 ms P99	1.8 万次搜索，平均 42 个候选
AI 落点搜索（2-ply）	0.213 ms 均值 / 0.94 ms P99	2.1 万次搜索，最坏 10.7 ms（首块 JIT）
单元测试	88 个 / 1.25 秒	反射直接驱动 xUnit 用例

表 4-3 AI 在真引擎上的长跑成绩

配置	平均分	平均消行	平均存活帧	顶死局数
1-ply	59330	593.3	8713	3 / 10
2-ply（默认）	68730	687.3	10000（上限）	0 / 10

0.213 ms 只占 16.67 ms 帧预算的 1.3%，观战不会拖慢界面；2-ply 的 10 局全部跑满上限仍未顶死，68730 分是下限而非真实上限。单帧逻辑不足 1 微秒，说明瓶颈完全在渲染侧，这也是我们把重心放在重绘节流而非逻辑优化的原因。

第五章　开发过程与反思

5.1　协作方式：接口先行

三个人从第一天起按“契约”而不是按文件分工：先定下五个接口、共享数据与依赖方向，再各自实现。规则还没写完时，界面已能用假引擎把窗口跑起来，AI 也已能用假规则跑通状态机。好处是几乎不互相阻塞，代价是接口一旦定错会波及三方——LockActive 最初顺手返回了“下一个方块”，AI 接入后发现推演被污染，才把出块职责移回引擎。

5.2　开发日志

仓库共 48 次提交，分布在 2026 年 10 月 25 日至 30 日共 6 天，覆盖骨架、规则、AI、界面接线、文档与发布。提交信息统一为“类型(范围): 说明”，关键修复能直接追到问题，例如“fix(ai): 落点枚举允许包围盒左列为负”。完整日志见 Git 记录文件。

5.3　定位问题的方法

最省时间的是固定下来的四步：先用一个失败的最小用例复现，再把问题缩到最小范围，然后用断言把“正确的行为”写成测试，最后才改实现并跑全量测试。左墙盲区那次恰恰因为先写了不变量测试而被骗过——不变量对了，覆盖度不够。

5.4　AI 工具的使用与边界

本次开发使用了 AI 编码助手（DeepSeek、Claude）辅助，主要用在四处：接口草案与注释初稿、样板代码（WinForms 布局、XML 序列化包装）、边界用例枚举、文档措辞。核心算法由我们自己推导并逐行核对，因为答辩要现场改代码。

AI 的输出有三类问题被否决或改写。第一类是“看起来对但偏离需求”：最初采用的计分写法参考了商业俄罗斯方块的等级倍率与硬降奖励，会让“一次消 k 行得 100 × k”对不上账，我们改回了严格 100 × k。第二类是“局部最优”：2-ply 全量加权正是它建议的写法，代码合理，实测 380 帧就顶死；发现方式是跑基准而不是读代码——改完分数不升反降就必须回退。第三类是“过度设计”：它建议引入依赖注入容器与通用“游戏模式”抽象，但本作业只有一种模式、五个对象，容

器只会把那 37 行变成三层注册；“把渲染抽象成 IRenderer”也被否决，WinForms 只有一个绘制入口，多一层接口没有调用方。

AI 的错误分两类：一类是一眼可见的笔误（少写一个格子、缩进错位），编译器与单测就能拦住；另一类是不报错但方向错的偏差，例如落点枚举从 0 开始、2-ply 全量加权——它们都能编译、都能跑，只会让 AI 变笨。后者更危险，只能靠具体的棋形用例和基准数据发现，这也是我们后来先在测试里写“竖 I 应覆盖每一列”再接引擎的原因。

5.5 反思

如果重来一次，有三处会改。第一，踢墙表应按“朝向对”写全，而不是一档通用偏移：现在能玩，但贴墙旋转的手感差一点，这是唯一一处为了省事牺牲手感的地方。第二，AI 权重是题目给的固定值，没有做网格调参。第三，测试覆盖 88 个用例，但没有统计行覆盖率，给不出“核心规则 90%+”这样的数字。

另一条体会：注释越像契约，协作越顺，但注释也越容易过期。我们把“为什么这么做”写进注释，把“做了什么”交给测试。4.2.4 那条“旋转态左移一列”正是靠注释与文档，才没在验收前夜去改一个不该改的地方。

第六章 结论

我们用 .NET 10 + WinForms 完成了一个可玩、可测、抗异常的俄罗斯方块：12 列 × 16 行棋盘、七种方块、双向旋转与简化踢墙、100 × k 计分与十六进制显示、XML 最高分持久化，以及一个稳定跑到 68730 分的启发式 AI。工程拆成五个单向依赖的项目，核心逻辑不依赖任何界面技术，88 个用例覆盖碰撞、旋转、消行、计分、引擎时序、存档异常与 AI。

最大的收获不是“写完了俄罗斯方块”，而是三件更通用的东西：接口先行使并行开发成为可能；“契约不抛异常”让异常处理从散落的 try/catch 变成一处集中的状态机；量化数据能立刻证伪一个看起来正确的改动。第四章那几个 bug，没有一个是被读代码发现的，全部是被测试或基准数据顶出来的。

后续改进按优先级排列：补全 SRS 朝向表；对 AI 权重做网格调参并引入更深的搜索；增加录制回放（种子与操作序列已具备可复现的基础）；把存档升级成 SQLite 排行榜；统计行覆盖率并补上界面层的自动化测试。

参考文献

[1] Microsoft. Windows Forms Timer 类 [EB/OL].
https://learn.microsoft.com/dotnet/api/system.windows.forms.timer

[2] Microsoft. XmlSerializer 类 [EB/OL].
https://learn.microsoft.com/dotnet/api/system.xml.serialization.xmlserializer

[3] Microsoft. .NET 应用程序发布概述 [EB/OL].
https://learn.microsoft.com/dotnet/core/deploying/

[4] xUnit.net. Getting Started with xUnit.net (CLI) [EB/OL].
https://xunit.net/docs/getting-started/netcore/cmdline

[5] Tetris Wiki. Tetris Guideline [EB/OL]. https://tetris.wiki/Tetris_Guideline

[6] Tetris Wiki. Super Rotation System [EB/OL].
https://tetris.wiki/Super_Rotation_System

[7] Tetris Wiki. Random Generator (7-bag) [EB/OL].
https://tetris.wiki/Random_Generator

附录 A　AI 使用声明

本组在本次大作业中使用了 AI 工具辅助开发，如实声明如下：所用工具为 DeepSeek 与 Claude（对话式编码助手）。AI 参与的环节包括接口草案与注释初稿、WinForms 布局与 XML 序列化的样板代码、边界测试用例的枚举、文档措辞整理。核心算法（碰撞、旋转踢墙、锁定消行、落点搜索、AI 指令状态机）由组员自行推导、逐行核对，并承担现场解释责任。

表 A-1　AI 使用情况明细

工具	使用位置	采纳情况	被否决 / 修改的内容
DeepSeek	接口草案、注释、任务书结构	部分采纳	过度抽象（容器、IRenderer）被去掉
DeepSeek	计分与等级曲线初稿	修改	初稿带等级倍率与硬降奖励，改为严格 100×k
Claude	WinForms 布局与自绘样板代码	采纳后重构	布局改为 TableLayoutPanel 两栏
Claude	2-ply 落点加权方案	否决	全量加权导致 380 帧顶死，改为并列最优裁决
Claude	测试用例边界清单	采纳	逐条改写为可直接运行的 xUnit 用例

所有 AI 生成的代码都经过组员阅读、改写与测试验证后才进入仓库；提交记录中的日期与内容反映真实开发顺序。验收时如被问到任意一段代码，我们可以说明它的写法与取舍。

附录 B　复现与验证

环境要求：Windows 10 / 11，.NET SDK 10.0（dotnet --version 显示 10.0.x）。

```
dotnet build Tetris.sln
dotnet run --project src/Tetris.App
dotnet test
```

发布（目标机器需安装 .NET 10 桌面运行时）：

```
dotnet publish src/Tetris.App -c Release -r win-x64 --self-contained false -o publish
```

存档位于 %AppData%\TetrisCourse\highscore.xml。删除后启动应提示“首次运行，暂无存档”并正常开局；改成乱码后启动应提示“存档损坏，已忽略”。这两步就是演示视频里的异常场景。